FPT UNIVERSITY HO CHI MINH CITY CAMPUS
FACULTY OF SOFTWARE ENGINEERING — EMBEDDED LINUX SYSTEMS

TECHNICAL REPORT — ASSIGNMENT 01
HOW TO WRITE A DEVICE DRIVER
(ioctl Interface | Misc Device | Driver Analysis | ARMv7 Target)

Student Full Name	Tran Gia Phuc
Student ID (MSSV)	SE204417
Course / Assignment	Embedded Linux Systems — ASSIGNMENT-01 (Chapters 8, 9, 10)
Development Host	Ubuntu Linux (Dell Inspiron 14 5430) — Cross-compiler: arm-linux-gnueabihf-gcc 15.2.0
Execution Target	QEMU ARMv7 Versatile Express Cortex-A9 (vexpress-a9), 256MB RAM
Kernel Version	Linux Kernel 5.15.0 SMP mod_unload ARMv7 p2v8
Implemented Drivers	1. asgn1_driver.ko (/dev/asgn1, major=241, 4 ioctl commands) 2. counter_driver.ko (/dev/counter, Misc major=10, minor=125, atomic_t)

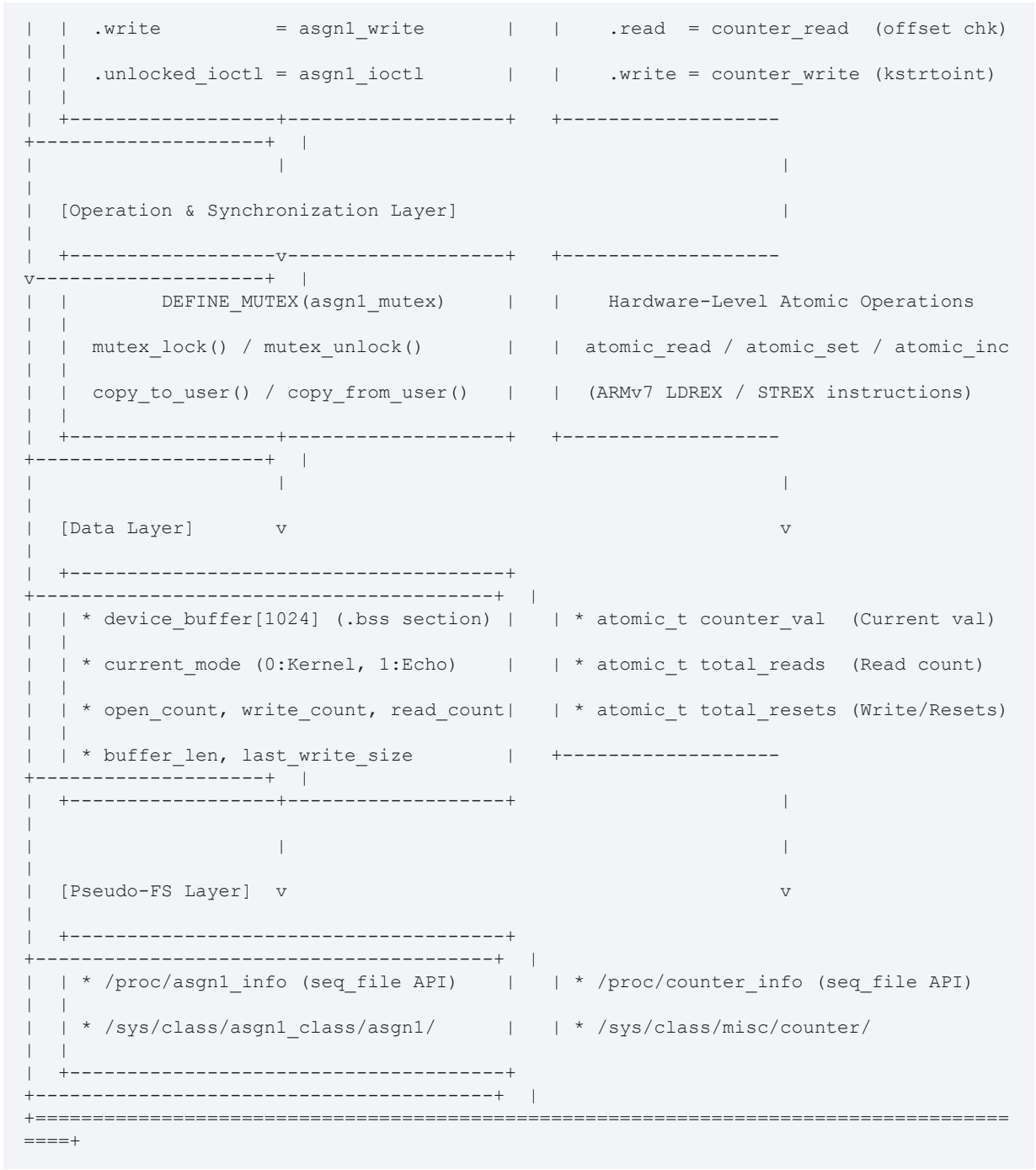
1. SECTION 1 (TASK 3): OVERALL ARCHITECTURE AND DATA FLOW

The system developed in Assignment 01 incorporates two independent kernel-space device drivers executing on the ARMv7 target architecture (vexpress-a9): (1) asgn1_driver, which manages a character device node /dev/asgn1 using traditional register_chrdev registration coupled with an ioctl control interface and mutex synchronization; and (2) counter_driver, which manages a simple hardware counter device node /dev/counter utilizing the lightweight miscdevice framework (Major=10) and atomic_t variables. The diagram below illustrates the comprehensive relationship among User Space applications, System Call boundaries, Virtual

File System (VFS), file operations / proc operations tables, internal data layers, and pseudo-filesystem interfaces (procfs/sysfs):

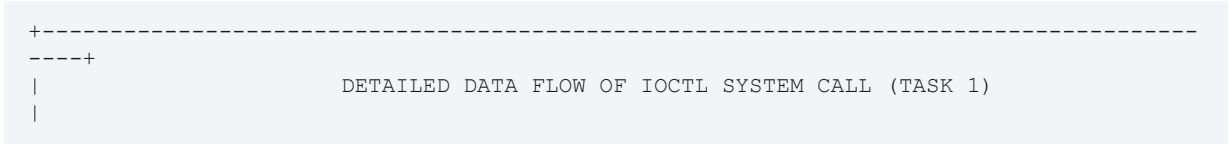
Diagram 1.1: Overall System Architecture integrating asgn1_driver and counter_driver





In addition to standard sequential stream reading/writing via read() and write(), the ioctl interface in asgn1_driver establishes a versatile control channel capable of transferring complex structured data and modifying device operational states dynamically. Diagram 1.2 details the execution lifecycle of an ioctl() system call originating from ioctl_app, traversing into asgn1_ioctl() within kernel space, and returning back to user space:

Diagram 1.2: Execution Flow of ioctl() System Call and Boundary Data Transfer





2. SECTION 2 (TASK 3): COMPARISON OF THREE LINUX DEVICE DRIVER TYPES

Within the Linux kernel architecture, hardware device drivers are categorized into three primary classes based on data stream characteristics and access methodologies: Character Devices, Block Devices, and Network Devices. Table 2.1 summarizes a rigorous technical comparison across five core evaluation criteria:

Evaluation Criteria	Character Device Driver	Block Device Driver	Network Device Driver
1. VFS / Kernel Subsystem Integration	Interfaces directly with VFS via struct file_operations (open, read, write, unlocked_ioctl, mmap, release). Represented by a character device node ('c') in /dev.	Interfaces through struct block_device_operations coupled with a Request Queue / blk-mq. Represented by a block device node ('b') in /dev, typically formatted and mounted as a File System (ext4, FAT).	Bypasses VFS node creation in /dev entirely. Registers directly with the Network Subsystem via struct net_device and struct net_device_ops (ndo_open, ndo_start_xmit, ndo_do_ioctl).
2. Data Transfer Unit	Sequential byte stream of arbitrary length ranging from 1 byte to multiple kilobytes, processed immediately per read()/write() system call.	Fixed-size data blocks (Sectors, typically 512 bytes or 4096 bytes). Supports random access reading and writing at any Logical Block Addressing (LBA) sector.	Variable-length network packets or frames (bounded by MTU, e.g., 1500 bytes for Ethernet), managed via struct sk_buff (Socket Buffer) memory structures.
3. Buffering & Caching Mechanisms	Does not utilize system Page Cache. The driver independently manages internal buffers (e.g., static device_buffer[1024] or kfifo) or directly interacts with hardware registers.	Leverages Kernel Page Cache and I/O Schedulers (such as mq-deadline, kyber) to merge, reorder, and cache disk requests to optimize overall storage throughput.	Utilizes asynchronous RX/Ring buffers coupled with Direct Memory Access (DMA) and NAPI polling mechanisms to mitigate interrupt overhead under heavy packet traffic.
4. Embedded System Real-world Examples	- Serial UART communication ports (/dev/ttyAMA0, /dev/ttyS0). - GPIO controllers, I2C/SPI busses, temperature sensors, push buttons, character LCDs. - Hardware counter (/dev/counter), Watchdog timer (/dev/watchdog).	- MicroSD cards and eMMC flash storage (/dev/mmcblk0). - NVMe Solid State Drives (/dev/nvme0n1), USB Flash Drives (/dev/sda). - RAM disk storage (/dev/ram0).	- Integrated Ethernet MAC controllers (SMSC911x eth0 on vexpress-a9). - Wi-Fi modules (wlan0), industrial CAN bus interfaces (can0), Bluetooth PAN.
5. When to Choose	When the device streams data sequentially, requires real-time response latency, does not host a filesystem, and requires specialized	When the device provides high-capacity mass storage supporting random block access and requires structured filesystems (ext4,	When the device handles packetized communication protocols (TCP/IP, UDP, ARP, CAN stack) and interacts with user-space

	hardware register control via ioctl.	FAT32) for durable file persistence.	applications via Socket interfaces.

Two Concrete Design Decision Examples in Embedded Engineering:

• Design Example 1 (Laser/Lidar Distance Sensor communicating via I2C/UART on an Autonomous Mobile Robot): The embedded engineer must select a Character Device Driver (or Misc Device Driver). Rationale: Distance measurement samples must be read immediately by user applications in a streaming fashion (tens of bytes every 10ms cycle) without being cached by Block Layer subsystems, while control applications require ioctl() to configure sampling frequencies and power-saving operational modes dynamically.

• Design Example 2 (16GB eMMC Flash Storage hosting Operating System Images and Automotive Blackbox Databases): The engineer must select a Block Device Driver. Rationale: Data must be organized into partitioned storage with journaling filesystems (such as ext4), heavily utilizing Page Cache and I/O Schedulers to aggregate small write operations into contiguous 4KB blocks, thereby maximizing storage throughput and preserving flash memory endurance.

3. SECTION 3 (TASK 3): DESIGN ANALYSIS OF THE TWO DRIVERS

3.1. Why Task 1 Uses register_chrdev() while Task 2 Uses misc_register()

During the development of Assignment 01, two distinct driver registration mechanisms were deliberately chosen to highlight the engineering trade-off between complete structural flexibility and code conciseness:

1. The register_chrdev() Mechanism in Task 1 (asgn1_driver):

• Core Operation: Allocates a dedicated major number (Major Number = 241) exclusively for the driver, permitting the management of up to 256 minor numbers (0 to 255). To achieve automatic device node creation under devtmpfs/udev, the developer must explicitly invoke class_create() and device_create() during initialization, and unwind them in reverse order during exit.

• Advantages: Highly suitable for driver architectures managing multiple homogeneous hardware instances sharing a single major number but distinguished by minor numbers (e.g., multiple serial ports /dev/ttyS0, /dev/ttyS1...), or when establishing a dedicated device hierarchy class under /sys/class/asgn1_class/.

• Disadvantages: Boilerplate initialization and cleanup code is verbose (~30 lines of error-handling scaffolding), and consumes a full major number slot in the kernel's character device registry even if only one instance exists.

2. The misc_register() Mechanism in Task 2 (counter_driver):

• Core Operation: The Miscellaneous Device framework consolidates miscellaneous hardware

devices under a single fixed major number, MISC_MAJOR (10), distinguishing individual drivers via minor numbers (utilizing MISC_DYNAMIC_MINOR to let the kernel automatically assign an available minor number, 125). With a single misc_register(&counter_misc_device) call, the kernel automatically attaches the device to /sys/class/misc/counter, registers it within /proc/misc, and prompts devtmpfs to create /dev/counter (10, 125).

• Advantages: Conserves kernel major number resources, eliminates up to 70% of boilerplate initialization code (no manual class_create/device_create required), and drastically reduces resource leakage risks upon failure.

• Disadvantages: Restricted strictly to single-instance hardware devices; unsuitable if a driver must manage numerous sub-devices across contiguous minor number ranges.

3.2. Why Task 2 Uses atomic_t Instead of Standard int? Detailed Race Condition Analysis

In counter_driver (Task 2), the counter state variable is declared as static atomic_t counter_val = ATOMIC_INIT(0); and manipulated via atomic_read(), atomic_set(), and atomic_inc() instead of standard integer variables. This design choice is mandated by Symmetric Multiprocessing (SMP) architectures (vexpress-a9 features up to 4 Cortex-A9 processing cores) and Kernel Preemption policies.

In the C programming language, a standard increment operation (counter_val++) appears atomic in source code, but when compiled into ARMv7 machine instructions, it decomposes into three discrete Read-Modify-Write assembly steps:

(1) LDR r0, [r1] ; Load current value of counter_val from RAM memory into CPU register r0

(2) ADD r0, r0, #1 ; Increment the value inside register r0 by 1 unit

(3) STR r0, [r1] ; Store the updated value from register r0 back into RAM memory

Diagram 3.1: Illustration of Data Race Condition (Lost Update) at ARMv7 Register Level without atomic_t

```
DATA RACE CONDITION SCENARIO (LOST UPDATE) WHEN USING STANDARD 'int' VARIABLES:
Assume initial value in RAM: counter_val = 10.
Two concurrent processes P1 (on CPU Core 0) and P2 (on CPU Core 1) invoke
read("/dev/counter"):

Timeline | CPU Core 0 (Process P1)                | CPU Core 1 (Process P2)
| RAM (counter_val)
---------+----------------------------------------
+----------------------------------------+-----------------
  t = 0  | (Enter counter_read function)          | (Enter counter_read function)
|        10
  t = 1  | LDR r0, [counter_val]  --> r0 = 10     |
|        10
  t = 2  |                                        | LDR r0, [counter_val]  --> r0 = 10
|        10
  t = 3  | ADD r0, r0, #1         --> r0 = 11     | ADD r0, r0, #1         --> r0 = 11
|        10
  t = 4  | STR r0, [counter_val]  (Store 11 to RAM|
|        11
  t = 5  |                                        | STR r0, [counter_val]  (Overwrite
```

```
11!) |        11 (CORRUPTED!)

=> CONSEQUENCE: Although two read operations occurred, both P1 and P2 return the exact
same value "10",
   and the final counter value increments to 11 instead of 12 (Lost Update Bug)!
```

The Solution via atomic_t: On ARMv7 architectures, atomic_inc(&counter_val) invokes specialized hardware synchronization instructions, namely LDREX (Load Exclusive) and STREX (Store Exclusive), coordinated with the memory controller's Exclusive Monitor. If CPU Core 1 interrupts memory between CPU Core 0's load and store, the STREX instruction detects the bus contention, aborts the write, and automatically retries the cycle until atomic success is guaranteed. Consequently, counter increments achieve absolute atomicity without incurring the heavy context-switch overhead associated with Mutex sleeping locks.

3.3. Explanation of ioctl Magic Number and Decoding _IOC_NR, _IOC_SIZE from asgn1_ioctl.h

1. Why is an ioctl Magic Number Required?

The ioctl(int fd, unsigned int cmd, unsigned long arg) system call accepts an arbitrary file descriptor as its first argument. If device drivers assigned simple sequential command numbers (0, 1, 2, 3...), a user application bug passing an incorrect file descriptor (such as a serial port /dev/ttyAMA0 or block storage fd) into ioctl() would cause the recipient driver to interpret the command number as valid and execute unintended hardware operations (such as resetting buffers or altering baud rates), inducing severe data corruption. To eliminate this risk, the Linux kernel standardizes all ioctl command codes into a 32-bit integer partitioned into four distinct bitfields, where the Magic Number (8-bit Type field) acts as a unique cryptographic signature for each specific driver.

Diagram 3.2: 32-bit ioctl Command Code Register Layout and Decoding Macros in asgn1_ioctl.h

```
32-BIT IOCTL COMMAND CODE LAYOUT IN LINUX KERNEL (<asm-generic/ioctl.h>):

 31      30 29                           16 15                    8 7
0
+----------+-------------------------------+-----------------------
+------------------------+
|   DIR    |             SIZE              |     TYPE (MAGIC)      |      NR (NUMBER)
|
| (2 bits) |           (14 bits)           |       (8 bits)        |       (8 bits)
|
+----------+-------------------------------+-----------------------
+------------------------+
  Direction  Parameter payload size in bytes Driver Signature Magic:  Sequential
command ID:
  00: _IO    calculated via sizeof(type)     'A' = 0x41 (65)         1: RESET_BUFFER
  01: _IOW   (Max 16383 bytes)                                       2: GET_STATS
  10: _IOR                                                           3: SET_MODE
  11: _IOWR                                                          4: GET_VERSION

Macro Formula for Command Generation:
  _IOC(dir, type, nr, size) = ((dir) << 30) | ((size) << 16) | ((type) << 8) | ((nr)
<< 0)

Decoding Macros in Kernel Space (asgn1_driver.c):
```

```
  _IOC_DIR(cmd)  = ((cmd) >> 30) & 0x03
  _IOC_SIZE(cmd) = ((cmd) >> 16) & 0x3FFF
  _IOC_TYPE(cmd) = ((cmd) >> 8)  & 0xFF     <-- Validation: if (_IOC_TYPE(cmd) != 'A')
return -ENOTTY;
  _IOC_NR(cmd)   = ((cmd) >> 0)  & 0xFF
```

2. Concrete Hexadecimal Calculation, _IOC_NR, and _IOC_SIZE for all 4 Commands in asgn1_ioctl.h (with ASGN1_IOC_MAGIC = 'A' = 0x41):

• Command 1: ASGN1_RESET_BUFFER = _IO('A', 1)
- Direction (_IOC_DIR) = 0 (00b - No data payload transfer)
- Size (_IOC_SIZE) = 0 bytes (0x0000)
- Type (_IOC_TYPE) = 'A' = 0x41 (65)
- Number (_IOC_NR) = 1 (0x01)
=> 32-bit Command Code = (0 << 30) | (0 << 16) | (0x41 << 8) | 0x01 = 0x00004101.

• Command 2: ASGN1_GET_STATS = _IOR('A', 2, struct asgn1_stats)
- Direction (_IOC_DIR) = 2 (10b - Read from Kernel to User Space)
- Size (_IOC_SIZE) = sizeof(struct asgn1_stats) = 5 * sizeof(int) = 20 bytes (0x0014)
- Type (_IOC_TYPE) = 'A' = 0x41
- Number (_IOC_NR) = 2 (0x02)
=> 32-bit Command Code = (2 << 30) | (0x0014 << 16) | (0x41 << 8) | 0x02 = 0x80144102.

• Command 3: ASGN1_SET_MODE = _IOW('A', 3, int)
- Direction (_IOC_DIR) = 1 (01b - Write from User Space to Kernel)
- Size (_IOC_SIZE) = sizeof(int) = 4 bytes (0x0004)
- Type (_IOC_TYPE) = 'A' = 0x41
- Number (_IOC_NR) = 3 (0x03)
=> 32-bit Command Code = (1 << 30) | (0x0004 << 16) | (0x41 << 8) | 0x03 = 0x40044103.

• Command 4: ASGN1_GET_VERSION = _IOR('A', 4, char[64])
- Direction (_IOC_DIR) = 2 (10b - Read from Kernel to User Space)
- Size (_IOC_SIZE) = sizeof(char[64]) = 64 bytes (0x0040)
- Type (_IOC_TYPE) = 'A' = 0x41
- Number (_IOC_NR) = 4 (0x04)
=> 32-bit Command Code = (2 << 30) | (0x0040 << 16) | (0x41 << 8) | 0x04 = 0x80404104.

3.4. Implemented Source Code Listings

Source Code 3.1: Shared Header task1_ioctl/asgn1_ioctl.h

```
/*
 * File: asgn1_ioctl.h
 * Author: Tran Gia Phuc - SE204417
 * Description: Shared header file between Kernel Space (asgn1_driver)
 *              and User Space for ioctl command definitions and data structures.
```

```
 */

#ifndef ASGN1_IOCTL_H
#define ASGN1_IOCTL_H

#ifdef __KERNEL__
#include <linux/ioctl.h>
#include <linux/types.h>
#else
#include <sys/ioctl.h>
#include <stdint.h>
#endif

/*
 * Magic Number ('A' = 0x41) giúp phân biệt tập lệnh ioctl của asgn1_driver
 * với các thiết bị khác trong hệ thống, ngăn chặn lỗi gửi nhầm lệnh
 * vào sai file descriptor.
 */
#define ASGN1_IOC_MAGIC        'A'

/* Các chế độ hoạt động của driver cho lệnh ASGN1_SET_MODE */
#define ASGN1_MODE_KERNEL      0   /* Tự động thêm tiền tố "[KERNEL] " khi write */
#define ASGN1_MODE_ECHO        1   /* Lưu dữ liệu nguyên bản (Mặc định) */

/* Kích thước tối đa của chuỗi version trả về cho userspace */
#define ASGN1_VERSION_MAX_LEN  64

/*
 * Cấu trúc thống kê hoạt động của driver (dùng cho ASGN1_GET_STATS).
 * Chứa đầy đủ 5 trường bắt buộc theo yêu cầu đề bài.
 */
struct asgn1_stats {
    int open_count;       /* Tổng số lần thiết bị được mở */
    int write_count;      /* Số lần ghi (reset về 0 khi gọi ASGN1_RESET_BUFFER) */
    int read_count;       /* Số lần đọc (reset về 0 khi gọi ASGN1_RESET_BUFFER) */
    int buffer_len;       /* Độ dài dữ liệu hiện tại trong buffer (bytes) */
    int last_write_size;  /* Kích thước dữ liệu của lần write gần nhất (bytes) */
};

/*
 * Định nghĩa 4 lệnh ioctl bằng các macro chuẩn của Linux Kernel:
 * 1. ASGN1_RESET_BUFFER : Không truyền tham số (_IO)
 * 2. ASGN1_GET_STATS    : Đọc struct asgn1_stats từ kernel về userspace (_IOR)
 * 3. ASGN1_SET_MODE     : Ghi cấu hình mode (kiểu int) từ userspace xuống kernel
(_IOW)
 * 4. ASGN1_GET_VERSION  : Đọc chuỗi version (mảng 64 bytes) từ kernel về userspace
(_IOR)
 */
#define ASGN1_RESET_BUFFER  _IO(ASGN1_IOC_MAGIC, 1)
#define ASGN1_GET_STATS     _IOR(ASGN1_IOC_MAGIC, 2, struct asgn1_stats)
#define ASGN1_SET_MODE      _IOW(ASGN1_IOC_MAGIC, 3, int)
#define ASGN1_GET_VERSION   _IOR(ASGN1_IOC_MAGIC, 4, char[ASGN1_VERSION_MAX_LEN])

/* Giới hạn số thứ tự lệnh tối đa để kiểm tra tính hợp lệ nếu cần */
#define ASGN1_IOC_MAXNR     4

#endif /* ASGN1_IOCTL_H */
```

Source Code 3.2: Character Driver task1_ioctl/asgn1_driver.c

```
/*
 * File: asgn1_driver.c
 * Author: Tran Gia Phuc - SE204417
 * Description: ASSIGNMENT-01 Task 1 - Extended Character Device Driver
 *              supporting 4 ioctl commands, mode switching, statistics,
 *              procfs/sysfs interfaces, and mutex synchronization.
```

```
 */

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/uaccess.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/proc_fs.h>
#include <linux/seq_file.h>
#include <linux/slab.h>
#include <linux/mutex.h>
#include <linux/string.h>

#include "asgn1_ioctl.h"

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Tran Gia Phuc - SE204417");
MODULE_DESCRIPTION("ASSIGNMENT-01: Character Device Driver with ioctl Interface");
MODULE_VERSION("1.0");

#define DEVICE_NAME     "asgn1"
#define CLASS_NAME      "asgn1_class"
#define BUFFER_SIZE     1024
#define DRIVER_MAJOR    241     /* Nằm trong khoảng 241-254 theo yêu cầu đề bài */
#define KERNEL_PREFIX   "[KERNEL] "
#define DRIVER_VER_STR  "asgn1_driver v1.0 - SE204417"

/* ===== DATA LAYER (Tầng dữ liệu trạng thái của Driver) ===== */
static int    major_number    = DRIVER_MAJOR;
static char   device_buffer[BUFFER_SIZE];
static int    buffer_len      = 0;
static int    open_count      = 0;
static int    write_count     = 0;
static int    read_count      = 0;
static int    last_write_size = 0;
static int    current_mode    = ASGN1_MODE_ECHO; /* Mặc định là Mode 1 (Echo mode) */

static struct class  *asgn1_class  = NULL;
static struct device *asgn1_device = NULL;
static struct proc_dir_entry *proc_entry = NULL;

/*
 * Mutex bảo vệ toàn bộ thao tác đọc/ghi vào device_buffer và các biến counter
 * nhằm tránh xung đột dữ liệu (race condition) khi nhiều tiến trình truy cập.
 */
static DEFINE_MUTEX(asgn1_mutex);

/* ===== PROCFS INTERFACE (/proc/asgn1_info) ===== */

/*
 * Hàm asgn1_proc_show: Xuất thông tin trạng thái của driver ra /proc/asgn1_info
 * Parameters:
 *   @m: Con trỏ cấu trúc seq_file quản lý bộ đệm xuất
 *   @v: Con trỏ dữ liệu riêng (không sử dụng)
 * Return: 0 khi thành công
 */
static int asgn1_proc_show(struct seq_file *m, void *v)
{
    mutex_lock(&asgn1_mutex);
    seq_printf(m, "ASSIGNMENT-01 Driver Statistics\n");
    seq_printf(m, "================================\n");
    seq_printf(m, "Driver version  : %s\n", DRIVER_VER_STR);
    seq_printf(m, "Device node     : /dev/%s (major=%d)\n", DEVICE_NAME,
major_number);
    seq_printf(m, "Current mode    : %d (%s)\n", current_mode,
               (current_mode == ASGN1_MODE_KERNEL) ? "Kernel mode" : "Echo mode");
    seq_printf(m, "Open count      : %d\n", open_count);
    seq_printf(m, "Write count     : %d\n", write_count);
```

```
    seq_printf(m, "Read count      : %d\n", read_count);
    seq_printf(m, "Buffer length   : %d bytes\n", buffer_len);
    seq_printf(m, "Last write size : %d bytes\n", last_write_size);
    if (buffer_len > 0)
        seq_printf(m, "Buffer content  : [%s]\n", device_buffer);
    else
        seq_printf(m, "Buffer content  : (empty)\n");
    mutex_unlock(&asgn1_mutex);
    return 0;
}

/*
 * Hàm asgn1_proc_open: Mở file /proc/asgn1_info sử dụng single_open
 * Parameters:
 *   @inode: Con trỏ inode của file trong procfs
 *   @file:  Con trỏ cấu trúc file đang mở
 * Return: 0 nếu thành công, mã lỗi âm nếu thất bại
 */
static int asgn1_proc_open(struct inode *inode, struct file *file)
{
    return single_open(file, asgn1_proc_show, NULL);
}

static const struct proc_ops asgn1_proc_fops = {
    .proc_open    = asgn1_proc_open,
    .proc_read    = seq_read,
    .proc_lseek   = seq_lseek,
    .proc_release = single_release,
};

/* ===== FILE OPERATIONS LAYER (Tầng xử lý thao tác thiết bị) ===== */

/*
 * Hàm asgn1_open: Được gọi khi userspace thực hiện open("/dev/asgn1", ...)
 * Parameters:
 *   @inode: Thông tin siêu dữ liệu của device node
 *   @file:  Đối tượng phiên làm việc của file mở
 * Return: 0 (luôn thành công)
 */
static int asgn1_open(struct inode *inode, struct file *file)
{
    mutex_lock(&asgn1_mutex);
    open_count++;
    pr_info("asgn1_driver: device opened (open_count=%d)\n", open_count);
    mutex_unlock(&asgn1_mutex);
    return 0;
}

/*
 * Hàm asgn1_release: Được gọi khi userspace đóng file descriptor bằng close()
 * Parameters:
 *   @inode: Thông tin siêu dữ liệu của device node
 *   @file:  Đối tượng phiên làm việc của file đóng
 * Return: 0 (luôn thành công)
 */
static int asgn1_release(struct inode *inode, struct file *file)
{
    pr_info("asgn1_driver: device closed\n");
    return 0;
}

/*
 * Hàm asgn1_read: Đọc dữ liệu từ device_buffer trả về không gian người dùng
 * Parameters:
 *   @file:   Con trỏ cấu trúc file
 *   @buf:    Bộ đệm trong không gian người dùng (__user) nhận dữ liệu
 *   @len:    Số byte tối đa mà userspace yêu cầu đọc
 *   @offset: Con trỏ vị trí đọc hiện tại trong buffer
 * Return: Số byte thực tế đọc được (0 nếu chạm EOF), hoặc -EFAULT nếu lỗi vùng nhớ
```

```
 */
static ssize_t asgn1_read(struct file *file, char __user *buf,
                          size_t len, loff_t *offset)
{
    int bytes_read = 0;

    mutex_lock(&asgn1_mutex);
    if (*offset >= buffer_len) {
        mutex_unlock(&asgn1_mutex);
        return 0; /* Đã đọc hết dữ liệu (EOF) */
    }

    if (len > buffer_len - *offset)
        len = buffer_len - *offset;

    if (copy_to_user(buf, device_buffer + *offset, len)) {
        mutex_unlock(&asgn1_mutex);
        pr_err("asgn1_driver: copy_to_user failed in read\n");
        return -EFAULT;
    }

    *offset += len;
    bytes_read = len;
    read_count++;

    pr_info("asgn1_driver: read %d bytes (read_count=%d)\n",
            bytes_read, read_count);
    mutex_unlock(&asgn1_mutex);

    return bytes_read;
}

/*
 * Hàm asgn1_write: Nhận dữ liệu từ userspace và lưu vào device_buffer theo chế độ
 *                  - Mode 0 (Kernel mode): Tự động thêm tiền tố "[KERNEL] "
 *                  - Mode 1 (Echo mode)  : Lưu dữ liệu nguyên bản
 *                  Lưu ý: Ghi trực tiếp vào device_buffer (.bss) thay vì cấp phát
 *                  mảng 1024 bytes trên Kernel Stack để tránh lỗi tràn frame size.
 * Parameters:
 *   @file:   Con trỏ cấu trúc file
 *   @buf:    Bộ đệm dữ liệu nguồn từ userspace (__user)
 *   @len:    Số byte mà userspace muốn ghi
 *   @offset: Con trỏ vị trí ghi
 * Return: Số byte từ userspace đã xử lý, hoặc -EFAULT nếu lỗi copy_from_user
 */
static ssize_t asgn1_write(struct file *file, const char __user *buf,
                           size_t len, loff_t *offset)
{
    size_t copy_len = len;
    size_t prefix_len = strlen(KERNEL_PREFIX);
    size_t max_payload;

    mutex_lock(&asgn1_mutex);

    if (current_mode == ASGN1_MODE_KERNEL) {
        max_payload = (BUFFER_SIZE - 1) - prefix_len;
        if (copy_len > max_payload)
            copy_len = max_payload;

        memcpy(device_buffer, KERNEL_PREFIX, prefix_len);
        if (copy_from_user(device_buffer + prefix_len, buf, copy_len)) {
            mutex_unlock(&asgn1_mutex);
            pr_err("asgn1_driver: copy_from_user failed in write (kernel mode)\n");
            return -EFAULT;
        }
        buffer_len = prefix_len + copy_len;
    } else {
        max_payload = BUFFER_SIZE - 1;
        if (copy_len > max_payload)
```

```
            copy_len = max_payload;

        if (copy_from_user(device_buffer, buf, copy_len)) {
            mutex_unlock(&asgn1_mutex);
            pr_err("asgn1_driver: copy_from_user failed in write (echo mode)\n");
            return -EFAULT;
        }
        buffer_len = copy_len;
    }

    /* Xóa ký tự xuống dòng '\n' ở cuối (nếu ghi bằng lệnh echo) */
    if (buffer_len > 0 && device_buffer[buffer_len - 1] == '\n')
        buffer_len--;

    device_buffer[buffer_len] = '\0';
    write_count++;
    last_write_size = buffer_len;

    pr_info("asgn1_driver: write in mode=%d, stored %d bytes: [%s] (write_count=%d)
\n",
            current_mode, buffer_len, device_buffer, write_count);

    mutex_unlock(&asgn1_mutex);

    return len;
}

/*
 * Hàm asgn1_ioctl: Xử lý 4 lệnh điều khiển ioctl từ không gian người dùng
 * Parameters:
 *   @file: Con trỏ cấu trúc file đang mở
 *   @cmd:  Mã lệnh ioctl 32-bit được tạo bởi macro _IO/_IOR/_IOW
 *   @arg:  Tham số đi kèm (giá trị hoặc địa chỉ con trỏ vùng nhớ userspace)
 * Return: 0 nếu thành công, -ENOTTY nếu sai lệnh, -EINVAL nếu sai tham số,
 *         -EFAULT nếu lỗi truyền dữ liệu qua ranh giới User/Kernel
 */
static long asgn1_ioctl(struct file *file, unsigned int cmd, unsigned long arg)
{
    struct asgn1_stats stats;
    int new_mode;
    char ver_buf[ASGN1_VERSION_MAX_LEN];

    /* Kiểm tra Magic Number để đảm bảo lệnh thuộc về asgn1_driver */
    if (_IOC_TYPE(cmd) != ASGN1_IOC_MAGIC) {
        pr_warn("asgn1_driver: ioctl invalid magic 0x%x (expected 0x%x)\n",
                _IOC_TYPE(cmd), ASGN1_IOC_MAGIC);
        return -ENOTTY;
    }

    switch (cmd) {
    case ASGN1_RESET_BUFFER:
        mutex_lock(&asgn1_mutex);
        memset(device_buffer, 0, BUFFER_SIZE);
        buffer_len  = 0;
        write_count = 0;
        read_count  = 0;
        mutex_unlock(&asgn1_mutex);
        pr_info("asgn1_driver: ioctl ASGN1_RESET_BUFFER executed (buffer cleared,
write_count=0, read_count=0)\n");
        break;

    case ASGN1_GET_STATS:
        mutex_lock(&asgn1_mutex);
        stats.open_count      = open_count;
        stats.write_count     = write_count;
        stats.read_count      = read_count;
        stats.buffer_len      = buffer_len;
        stats.last_write_size = last_write_size;
        mutex_unlock(&asgn1_mutex);
```

```
        if (copy_to_user((struct asgn1_stats __user *)arg, &stats, sizeof(stats))) {
            pr_err("asgn1_driver: ioctl ASGN1_GET_STATS copy_to_user failed\n");
            return -EFAULT;
        }
        pr_info("asgn1_driver: ioctl ASGN1_GET_STATS executed (open=%d, write=%d,
read=%d, len=%d, last_write=%d)\n",
                stats.open_count, stats.write_count, stats.read_count,
                stats.buffer_len, stats.last_write_size);
        break;

    case ASGN1_SET_MODE:
        if (copy_from_user(&new_mode, (int __user *)arg, sizeof(int))) {
            pr_err("asgn1_driver: ioctl ASGN1_SET_MODE copy_from_user failed\n");
            return -EFAULT;
        }
        if (new_mode != ASGN1_MODE_KERNEL && new_mode != ASGN1_MODE_ECHO) {
            pr_warn("asgn1_driver: ioctl ASGN1_SET_MODE invalid mode=%d\n", new_mode);
            return -EINVAL;
        }
        mutex_lock(&asgn1_mutex);
        current_mode = new_mode;
        mutex_unlock(&asgn1_mutex);
        pr_info("asgn1_driver: ioctl ASGN1_SET_MODE set to %d (%s)\n",
                current_mode,
                (current_mode == ASGN1_MODE_KERNEL) ? "Kernel mode" : "Echo mode");
        break;

    case ASGN1_GET_VERSION:
        memset(ver_buf, 0, ASGN1_VERSION_MAX_LEN);
        strscpy(ver_buf, DRIVER_VER_STR, ASGN1_VERSION_MAX_LEN);
        if (copy_to_user((char __user *)arg, ver_buf, ASGN1_VERSION_MAX_LEN)) {
            pr_err("asgn1_driver: ioctl ASGN1_GET_VERSION copy_to_user failed\n");
            return -EFAULT;
        }
        pr_info("asgn1_driver: ioctl ASGN1_GET_VERSION executed (version=\"%s\")\n",
                ver_buf);
        break;

    default:
        pr_warn("asgn1_driver: ioctl unknown command 0x%x\n", cmd);
        return -ENOTTY;
    }

    return 0;
}

/* Bảng ánh xạ các hàm xử lý vào VFS (Virtual File System) */
static const struct file_operations asgn1_fops = {
    .owner          = THIS_MODULE,
    .open           = asgn1_open,
    .release        = asgn1_release,
    .read           = asgn1_read,
    .write          = asgn1_write,
    .unlocked_ioctl = asgn1_ioctl,
};

/* ===== MODULE INIT & EXIT LAYER ===== */

/*
 * Hàm asgn1_init: Khởi tạo và đăng ký character driver khi nạp bằng insmod
 * Return: 0 nếu thành công, mã lỗi âm nếu thất bại
 */
static int __init asgn1_init(void)
{
    int ret;

    pr_info("asgn1_driver: initializing module (version: %s)\n", DRIVER_VER_STR);
```

```
    ret = register_chrdev(major_number, DEVICE_NAME, &asgn1_fops);
    if (ret < 0) {
        pr_err("asgn1_driver: register_chrdev failed with error %d\n", ret);
        return ret;
    }

    asgn1_class = class_create(THIS_MODULE, CLASS_NAME);
    if (IS_ERR(asgn1_class)) {
        unregister_chrdev(major_number, DEVICE_NAME);
        return PTR_ERR(asgn1_class);
    }

    asgn1_device = device_create(asgn1_class, NULL,
                                 MKDEV(major_number, 0), NULL, DEVICE_NAME);
    if (IS_ERR(asgn1_device)) {
        class_destroy(asgn1_class);
        unregister_chrdev(major_number, DEVICE_NAME);
        return PTR_ERR(asgn1_device);
    }

    proc_entry = proc_create("asgn1_info", 0444, NULL, &asgn1_proc_fops);
    if (!proc_entry)
        pr_warn("asgn1_driver: failed to create /proc/asgn1_info\n");

    pr_info("asgn1_driver: loaded successfully on /dev/%s (major=%d, default_mode=%d)
\n",
            DEVICE_NAME, major_number, current_mode);
    return 0;
}

/*
 * Hàm asgn1_exit: Giải phóng toàn bộ tài nguyên khi gỡ module bằng rmmod
 */
static void __exit asgn1_exit(void)
{
    if (proc_entry)
        proc_remove(proc_entry);
    device_destroy(asgn1_class, MKDEV(major_number, 0));
    class_destroy(asgn1_class);
    unregister_chrdev(major_number, DEVICE_NAME);
    pr_info("asgn1_driver: module unloaded cleanly\n");
}

module_init(asgn1_init);
module_exit(asgn1_exit);
```

Source Code 3.3: Misc Device Driver task2_misc/counter_driver.c

```
/*
 * File: counter_driver.c
 * Author: Tran Gia Phuc - SE204417
 * Description: ASSIGNMENT-01 Task 2 - Miscellaneous Device Driver (Counter Device)
 *              Using miscdevice framework (major 10), atomic_t for thread-safety,
 *              auto-increment on read, kstrtoint parsing on write, and procfs stats.
 */

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/uaccess.h>
#include <linux/miscdevice.h>
#include <linux/atomic.h>
#include <linux/proc_fs.h>
#include <linux/seq_file.h>
#include <linux/string.h>
```

```
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Tran Gia Phuc - SE204417");
MODULE_DESCRIPTION("ASSIGNMENT-01: Miscellaneous Device Driver - Counter Device");
MODULE_VERSION("1.0");

#define DEVICE_NAME     "counter"
#define PROCFS_NAME     "counter_info"

/* ===== DATA LAYER (Tầng dữ liệu an toàn đa luồng SMP) ===== */
static atomic_t counter_val   = ATOMIC_INIT(0); /* Giá trị bộ đếm hiện tại */
static atomic_t total_reads   = ATOMIC_INIT(0); /* Tổng số lần đọc thành công */
static atomic_t total_resets  = ATOMIC_INIT(0); /* Tổng số lần ghi/reset bộ đếm */

static struct proc_dir_entry *proc_entry = NULL;

/* ===== PROCFS INTERFACE (/proc/counter_info) ===== */
static int counter_proc_show(struct seq_file *m, void *v)
{
    seq_printf(m, "ASSIGNMENT-01 Counter Driver Statistics\n");
    seq_printf(m, "=======================================\n");
    seq_printf(m, "Device name     : /dev/%s (Misc major: 10)\n", DEVICE_NAME);
    seq_printf(m, "Current counter : %d\n", atomic_read(&counter_val));
    seq_printf(m, "Total reads     : %d\n", atomic_read(&total_reads));
    seq_printf(m, "Total resets    : %d\n", atomic_read(&total_resets));
    return 0;
}

static int counter_proc_open(struct inode *inode, struct file *file)
{
    return single_open(file, counter_proc_show, NULL);
}

static const struct proc_ops counter_proc_fops = {
    .proc_open    = counter_proc_open,
    .proc_read    = seq_read,
    .proc_lseek   = seq_lseek,
    .proc_release = single_release,
};

/* ===== FILE OPERATIONS LAYER ===== */

/*
 * Hàm counter_read: Trả về giá trị hiện tại của counter dưới dạng chuỗi thập phân
 *                   và tự động tăng counter lên 1 (auto-increment on read).
 * Xử lý đúng *ppos (file offset) để lệnh cat không bị lặp vô tận.
 */
static ssize_t counter_read(struct file *file, char __user *buf,
                            size_t len, loff_t *offset)
{
    char tmp_str[32];
    int val, str_len;
    int bytes_copied;

    /* Nếu offset > 0, nghĩa là đã đọc ở lần gọi trước, trả về 0 (EOF) */
    if (*offset > 0)
        return 0;

    val = atomic_read(&counter_val);
    str_len = snprintf(tmp_str, sizeof(tmp_str), "%d\n", val);

    if (len < str_len)
        return -EINVAL;

    if (copy_to_user(buf, tmp_str, str_len)) {
        pr_err("counter_driver: copy_to_user failed in read\n");
        return -EFAULT;
    }

    *offset += str_len;
```

```
    bytes_copied = str_len;

    /* Tự động tăng counter lên 1 và tăng bộ đếm thống kê read */
    atomic_inc(&counter_val);
    atomic_inc(&total_reads);

    pr_info("counter_driver: read counter=%d, auto-incremented\n", val);
    return bytes_copied;
}

/*
 * Hàm counter_write: Nhận chuỗi ASCII từ userspace, phân tích thành số nguyên
 *                    bằng kstrtoint() để set hoặc reset giá trị counter.
 */
static ssize_t counter_write(struct file *file, const char __user *buf,
                             size_t len, loff_t *offset)
{
    char kbuf[32];
    int new_val, ret;
    size_t copy_len = len;

    if (copy_len > sizeof(kbuf) - 1)
        copy_len = sizeof(kbuf) - 1;

    if (copy_from_user(kbuf, buf, copy_len)) {
        pr_err("counter_driver: copy_from_user failed in write\n");
        return -EFAULT;
    }
    kbuf[copy_len] = '\0';

    /* Xóa ký tự xuống dòng '\n' nếu có */
    if (copy_len > 0 && kbuf[copy_len - 1] == '\n')
        kbuf[copy_len - 1] = '\0';

    /* Phân tích chuỗi thành số nguyên an toàn bằng kstrtoint */
    ret = kstrtoint(kbuf, 10, &new_val);
    if (ret < 0) {
        pr_warn("counter_driver: invalid non-numeric input: \"%s\"\n", kbuf);
        return -EINVAL; /* Không phải số hợp lệ */
    }

    if (new_val < 0) {
        pr_warn("counter_driver: negative value not allowed: %d\n", new_val);
        return -EINVAL; /* Giá trị âm */
    }

    /* Cập nhật giá trị counter và tăng số lần write/reset */
    atomic_set(&counter_val, new_val);
    atomic_inc(&total_resets);

    pr_info("counter_driver: counter set to %d via write\n", new_val);
    return len;
}

static const struct file_operations counter_fops = {
    .owner   = THIS_MODULE,
    .read    = counter_read,
    .write   = counter_write,
};

/* Đăng ký miscdevice cấu trúc chuẩn của kernel */
static struct miscdevice counter_misc_device = {
    .minor   = MISC_DYNAMIC_MINOR, /* Kernel tự cấp minor number */
    .name    = DEVICE_NAME,        /* Tạo tự động /dev/counter */
    .fops    = &counter_fops,
};

/* ===== MODULE INIT & EXIT ===== */
static int __init counter_init(void)
```

```
{
    int ret;

    pr_info("counter_driver: initializing miscellaneous device driver\n");

    /* Đăng ký misc device (thay thế register_chrdev phức tạp) */
    ret = misc_register(&counter_misc_device);
    if (ret) {
        pr_err("counter_driver: misc_register failed with error %d\n", ret);
        return ret;
    }

    /* Tạo file thông tin /proc/counter_info */
    proc_entry = proc_create(PROCFS_NAME, 0444, NULL, &counter_proc_fops);
    if (!proc_entry)
        pr_warn("counter_driver: failed to create /proc/%s\n", PROCFS_NAME);

    pr_info("counter_driver: loaded successfully on /dev/%s (minor=%d)\n",
            DEVICE_NAME, counter_misc_device.minor);
    return 0;
}

static void __exit counter_exit(void)
{
    if (proc_entry)
        proc_remove(proc_entry);
    misc_deregister(&counter_misc_device);
    pr_info("counter_driver: module unloaded cleanly\n");
}

module_init(counter_init);
module_exit(counter_exit);
```

4. SECTION 4 (TASK 3): EXPERIMENTAL QEMU TESTING RESULTS AND DEBUGGING LOGS

4.1. Cross-Compilation and modinfo Inspection

Both asgn1_driver.ko and counter_driver.ko alongside ioctl_app were successfully cross-compiled on the Ubuntu host workstation using arm-linux-gnueabihf-gcc targeting Linux Kernel 5.15.0 ARMv7:

Terminal Log 4.1: Verification of ELF ARMv7 Binary Format and modinfo Metadata

```
$make -C /home/phuc/embedded_lab1/kernel/linux-5.15
M=/home/phuc/embedded_asgn1/task1_ioctl ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-
modules$ arm-linux-gnueabihf-gcc -Wall -O2 -static -o ioctl_app ioctl_app.c
$ file asgn1_driver.ko ioctl_app
asgn1_driver.ko: ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV), with
debug_info, not stripped
ioctl_app:       ELF 32-bit LSB executable, ARM, EABI5 version 1 (GNU/Linux),
statically linked, not stripped
$ modinfo asgn1_driver.ko
filename:       /home/phuc/embedded_asgn1/task1_ioctl/asgn1_driver.ko
version:        1.0
description:    ASSIGNMENT-01: Character Device Driver with ioctl Interface
author:         Tran Gia Phuc - SE204417
license:        GPL
name:           asgn1_driver
vermagic:       5.15.0 SMP mod_unload ARMv7 p2v8
```

```
$ modinfo counter_driver.ko
filename:       /home/phuc/embedded_asgn1/task2_misc/counter_driver.ko
version:        1.0
description:    ASSIGNMENT-01: Miscellaneous Device Driver - Counter Device
author:         Tran Gia Phuc - SE204417
license:        GPL
name:           counter_driver
vermagic:       5.15.0 SMP mod_unload ARMv7 p2v8
```

[Optional Screenshot Space - IMG-01: IMG-01_task1_build_modinfo.png (Paste screenshot or image here if desired)]

Figure IMG-01: Compilation and modinfo verification output for asgn1_driver.ko and counter_driver.ko on Ubuntu Host

4.2. Complete Execution Log of test_asgn1.sh (Task 1)

The following log (logs/test_asgn1_output.txt) was captured during execution of test_asgn1.sh on the ARMv7 QEMU virtual machine. The script validates the entire lifecycle: insmod insertion, /dev/asgn1 creation (major=241), ASGN1_GET_VERSION validation, writing/reading under Mode 1 (Echo Mode) and Mode 0 (Kernel Mode auto-prepending '[KERNEL] '), statistical verification of all 5 fields via ASGN1_GET_STATS, buffer and counter clearing via ASGN1_RESET_BUFFER (verifying cat /dev/asgn1 returns an empty string), invalid magic number rejection returning -ENOTTY (errno=25), and clean rmmod removal:

Terminal Log 4.2: Complete Test Execution Output (logs/test_asgn1_output.txt) on ARMv7 QEMU

```
  ASSIGNMENT 01 - TASK 1: IOCTL DRIVER TEST SUITE
  Student: Tran Gia Phuc - SE204417
============================================================

[STEP 1] Loading asgn1_driver.ko and verifying /dev/asgn1 (major=241)...
asgn1_driver: loading out-of-tree module taints kernel.
asgn1_driver: initializing module (version: asgn1_driver v1.0 - SE204417)
asgn1_driver: loaded successfully on /dev/asgn1 (major=241, default_mode=1)
crw-rw-rw-    1 root     0         241,   0 Oct 19 12:16 [1;35m/dev/asgn1[m
asgn1_driver 16384 0 - Live 0x7f000000 (O)

[STEP 2] Testing ioctl Command 4: ASGN1_GET_VERSION...
asgn1_driver: device opened (open_count=1)
asgn1_driver: ioctl ASGN1_GET_VERSION executed (version="asgn1_driver v1.0 -
SE204417")
[USERSPACE] ASGN1_GET_VERSION: "asgn1_driver v1.0 - SE204417"
asgn1_driver: device closed

[STEP 3] Testing Write/Read in Default Mode (Mode 1 - Echo Mode)...
asgn1_driver: device opened (open_count=2)
asgn1_driver: write in mode=1, stored 24 bytes: [Hello_Echo_Mode_SE204417]
(write_count=1)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=3)
asgn1_driver: read 24 bytes (read_count=1)
asgn1_driver: device closed
  -> Data written : Hello_Echo_Mode_SE204417
  -> Data read    : Hello_Echo_Mode_SE204417
```

```
  [PASS] Echo Mode preserved original string.

[STEP 4] Testing ioctl Command 3: ASGN1_SET_MODE -> Mode 0 (Kernel Mode)...
asgn1_driver: device opened (open_count=4)
asgn1_driver: ioctl ASGN1_SET_MODE set to 0 (Kernel mode)
[USERSPACE] ASGN1_SET_MODE set to 0 (Kernel mode).
asgn1_driver: device closed
asgn1_driver: device opened (open_count=5)
asgn1_driver: write in mode=0, stored 28 bytes: [[KERNEL] Data_From_Userspace]
(write_count=2)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=6)
asgn1_driver: read 28 bytes (read_count=2)
asgn1_driver: device closed
  -> Data written : Data_From_Userspace
  -> Data read    : [KERNEL] Data_From_Userspace
  [PASS] Kernel Mode prepended '[KERNEL] ' prefix accurately.

[STEP 5] Switching back to Mode 1 (Echo Mode) via ASGN1_SET_MODE...
asgn1_driver: device opened (open_count=7)
asgn1_driver: ioctl ASGN1_SET_MODE set to 1 (Echo mode)
[USERSPACE] ASGN1_SET_MODE set to 1 (Echo mode).
asgn1_driver: device closed
asgn1_driver: device opened (open_count=8)
asgn1_driver: write in mode=1, stored 17 bytes: [Back_To_Echo_Mode] (write_count=3)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=9)
asgn1_driver: read 17 bytes (read_count=3)
asgn1_driver: device closed
  -> Data read    : Back_To_Echo_Mode

[STEP 6] Testing ioctl Command 2: ASGN1_GET_STATS (before reset)...
asgn1_driver: device opened (open_count=10)
asgn1_driver: ioctl ASGN1_GET_STATS executed (open=10, write=3, read=3, len=17,
last_write=17)
[USERSPACE] ASGN1_GET_STATS result:
  -> open_count      : 10
  -> write_count     : 3
  -> read_count      : 3
  -> buffer_len      : 17 bytes
  -> last_write_size : 17 bytes
asgn1_driver: device closed
  -> Checking /proc/asgn1_info:
ASSIGNMENT-01 Driver Statistics
==============================
Driver version  : asgn1_driver v1.0 - SE204417
Device node     : /dev/asgn1 (major=241)
Current mode    : 1 (Echo mode)
Open count      : 10
Write count     : 3
Read count      : 3
Buffer length   : 17 bytes
Last write size : 17 bytes
Buffer content  : [Back_To_Echo_Mode]

[STEP 7] Testing ioctl Command 1: ASGN1_RESET_BUFFER...
asgn1_driver: device opened (open_count=11)
asgn1_driver: ioctl ASGN1_RESET_BUFFER executed (buffer cleared, write_count=0,
read_count=0)
[USERSPACE] ASGN1_RESET_BUFFER executed successfully.
asgn1_driver: device closed
asgn1_driver: device opened (open_count=12)
asgn1_driver: device closed
  -> Output of 'cat /dev/asgn1' after reset: []
  [PASS] Buffer is completely empty after ASGN1_RESET_BUFFER.
  -> Checking stats after reset (note: cat just incremented read_count on empty buffer
or returned EOF):
asgn1_driver: device opened (open_count=13)
asgn1_driver: ioctl ASGN1_GET_STATS executed (open=13, write=0, read=0, len=0,
```

```
last_write=17)
[USERSPACE] ASGN1_GET_STATS result:
  -> open_count      : 13
  -> write_count     : 0
  -> read_count      : 0
  -> buffer_len      : 0 bytes
  -> last_write_size : 17 bytes
asgn1_driver: device closed

[STEP 8] Testing invalid ioctl command handling (-ENOTTY)...
asgn1_driver: device opened (open_count=14)
asgn1_driver: ioctl invalid magic 0x5a (expected 0x41)
[USERSPACE] Invalid ioctl correctly rejected: Inappropriate ioctl for device
(errno=25)
asgn1_driver: device closed

[STEP 9] Unloading asgn1_driver module and checking dmesg...
asgn1_driver: module unloaded cleanly
  [PASS] rmmod asgn1_driver succeeded.

[STEP 10] Kernel log (dmesg) for asgn1_driver:
asgn1_driver: loading out-of-tree module taints kernel.
asgn1_driver: initializing module (version: asgn1_driver v1.0 - SE204417)
asgn1_driver: loaded successfully on /dev/asgn1 (major=241, default_mode=1)
asgn1_driver: device opened (open_count=1)
asgn1_driver: ioctl ASGN1_GET_VERSION executed (version="asgn1_driver v1.0 -
SE204417")
asgn1_driver: device closed
asgn1_driver: device opened (open_count=2)
asgn1_driver: write in mode=1, stored 24 bytes: [Hello_Echo_Mode_SE204417]
(write_count=1)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=3)
asgn1_driver: read 24 bytes (read_count=1)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=4)
asgn1_driver: ioctl ASGN1_SET_MODE set to 0 (Kernel mode)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=5)
asgn1_driver: write in mode=0, stored 28 bytes: [[KERNEL] Data_From_Userspace]
(write_count=2)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=6)
asgn1_driver: read 28 bytes (read_count=2)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=7)
asgn1_driver: ioctl ASGN1_SET_MODE set to 1 (Echo mode)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=8)
asgn1_driver: write in mode=1, stored 17 bytes: [Back_To_Echo_Mode] (write_count=3)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=9)
asgn1_driver: read 17 bytes (read_count=3)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=10)
asgn1_driver: ioctl ASGN1_GET_STATS executed (open=10, write=3, read=3, len=17,
last_write=17)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=11)
asgn1_driver: ioctl ASGN1_RESET_BUFFER executed (buffer cleared, write_count=0,
read_count=0)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=12)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=13)
asgn1_driver: ioctl ASGN1_GET_STATS executed (open=13, write=0, read=0, len=0,
last_write=17)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=14)
```

```
asgn1_driver: ioctl invalid magic 0x5a (expected 0x41)
asgn1_driver: device closed
asgn1_driver: module unloaded cleanly

============================================================
  TASK 1 TEST SUITE COMPLETED SUCCESSFULLY!
```

4.3. Complete Execution Log of test_counter.sh and /proc/counter_info at Three Timepoints (Task 2)

The following log (logs/test_counter_output.txt) records execution of test_counter.sh within QEMU. Results demonstrate successful counter_driver registration within the miscdevice framework (Major=10, Minor=125), automatic counter increments upon reading (0 -> 1 -> 2), setting counter to 100 and resetting to 0 via echo commands, and rejection of negative values (-50) and non-numeric strings ('abc_invalid') with -EINVAL. Crucially, /proc/counter_info output was captured across three distinct timepoints: (1) Before Write (Counter=3, Reads=3, Resets=0); (2) After writing 100 and reading twice (Counter=102, Reads=5, Resets=1); and (3) After resetting to 0 and reading once (Counter=1, Reads=6, Resets=2):

Terminal Log 4.3: Complete Test Output (logs/test_counter_output.txt) including 3 procfs timepoints

```
  ASSIGNMENT 01 - TASK 2: MISC COUNTER DRIVER TEST SUITE
  Student: Tran Gia Phuc - SE204417
============================================================

[STEP 1] Loading counter_driver.ko and checking /dev/counter (major=10)...
counter_driver: initializing miscellaneous device driver
counter_driver: loaded successfully on /dev/counter (minor=125)
crw-rw-rw-    1 root     0          10, 125 Oct 19 12:23 [1;35m/dev/counter[m
counter_driver 16384 0 - Live 0x7f000000 (O)
125 counter

[STEP 2] Testing READ auto-increment on /dev/counter...
counter_driver: read counter=0, auto-incremented
counter_driver: read counter=1, auto-incremented
counter_driver: read counter=2, auto-incremented
  -> Read #1 (expected 0): 0
  -> Read #2 (expected 1): 1
  -> Read #3 (expected 2): 2
  [PASS] Auto-increment on read works accurately.

[STEP 3] Checking /proc/counter_info [Timepoint #1 - Before Write]:
ASSIGNMENT-01 Counter Driver Statistics
=======================================
Device name     : /dev/counter (Misc major: 10)
Current counter : 3
Total reads     : 3
Total resets    : 0

[STEP 4] Testing WRITE: setting counter to 100...
counter_driver: counter set to 100 via write
counter_driver: read counter=100, auto-incremented
counter_driver: read counter=101, auto-incremented
  -> First read after setting 100 (expected 100): 100
  -> Next read (expected 101)                   : 101
  [PASS] Counter set to 100 and incremented to 101.

[STEP 5] Checking /proc/counter_info [Timepoint #2 - After Write 100]:
```

```
ASSIGNMENT-01 Counter Driver Statistics
=======================================
Device name     : /dev/counter (Misc major: 10)
Current counter : 102
Total reads     : 5
Total resets    : 1

[STEP 6] Testing WRITE '0' to reset counter to 0...
counter_driver: counter set to 0 via write
counter_driver: read counter=0, auto-incremented
  -> Read immediately after reset (expected 0): 0
  [PASS] Counter reset to 0 accurately.

[STEP 7] Checking /proc/counter_info [Timepoint #3 - After Reset]:
ASSIGNMENT-01 Counter Driver Statistics
=======================================
Device name     : /dev/counter (Misc major: 10)
Current counter : 1
Total reads     : 6
Total resets    : 2

[STEP 8] Testing invalid WRITE inputs (-EINVAL check)...
  -> Writing negative value '-50' to /dev/counter:
counter_driver: negative value not allowed: -50
  [PASS] Negative value '-50' rejected with error (-EINVAL).
  -> Writing non-numeric string 'abc_invalid' to /dev/counter:
counter_driver: invalid non-numeric input: "abc_invalid"
  [PASS] Non-numeric string 'abc_invalid' rejected with error (-EINVAL).

[STEP 9] Unloading counter_driver and checking dmesg...
counter_driver: module unloaded cleanly
  [PASS] rmmod counter_driver succeeded.

[STEP 10] Kernel log (dmesg) for counter_driver:
counter_driver: initializing miscellaneous device driver
counter_driver: loaded successfully on /dev/counter (minor=125)
counter_driver: read counter=0, auto-incremented
counter_driver: read counter=1, auto-incremented
counter_driver: read counter=2, auto-incremented
counter_driver: counter set to 100 via write
counter_driver: read counter=100, auto-incremented
counter_driver: read counter=101, auto-incremented
counter_driver: counter set to 0 via write
counter_driver: read counter=0, auto-incremented
counter_driver: negative value not allowed: -50
counter_driver: invalid non-numeric input: "abc_invalid"
counter_driver: module unloaded cleanly

============================================================
  TASK 2 TEST SUITE COMPLETED SUCCESSFULLY!
```

4.4. Kernel dmesg Log after Executing Both Test Suites

Kernel diagnostic messages extracted from logs/dmesg_full.txt confirm that module loading, device opening/closing, data streaming, ioctl command processing, and exception handling for both drivers executed flawlessly and cleaned up without resource leakage upon removal:

Terminal Log 4.4: Kernel dmesg Log tracing asgn1_driver and counter_driver activity

```
Linux version 5.15.0 (phuc@phuc-Inspiron-14-5430) (arm-linux-gnueabihf-gcc (Ubuntu
15.2.0-16ubuntu1) 15.2.0, GNU ld (GNU Binutils for Ubuntu) 2.46) #1 SMP Sat Sep 12
14:42:42 +07 2026
OF: fdt: Machine model: V2P-CA9
asgn1_driver: loading out-of-tree module taints kernel.
```

```
asgn1_driver: initializing module (version: asgn1_driver v1.0 - SE204417)
asgn1_driver: loaded successfully on /dev/asgn1 (major=241, default_mode=1)
asgn1_driver: device opened (open_count=1)
asgn1_driver: ioctl ASGN1_GET_VERSION executed (version="asgn1_driver v1.0 -
SE204417")
asgn1_driver: device closed
asgn1_driver: device opened (open_count=2)
asgn1_driver: write in mode=1, stored 24 bytes: [Hello_Echo_Mode_SE204417]
(write_count=1)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=3)
asgn1_driver: read 24 bytes (read_count=1)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=4)
asgn1_driver: ioctl ASGN1_SET_MODE set to 0 (Kernel mode)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=5)
asgn1_driver: write in mode=0, stored 28 bytes: [[KERNEL] Data_From_Userspace]
(write_count=2)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=6)
asgn1_driver: read 28 bytes (read_count=2)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=7)
asgn1_driver: ioctl ASGN1_SET_MODE set to 1 (Echo mode)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=8)
asgn1_driver: write in mode=1, stored 17 bytes: [Back_To_Echo_Mode] (write_count=3)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=9)
asgn1_driver: read 17 bytes (read_count=3)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=10)
asgn1_driver: ioctl ASGN1_GET_STATS executed (open=10, write=3, read=3, len=17,
last_write=17)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=11)
asgn1_driver: ioctl ASGN1_RESET_BUFFER executed (buffer cleared, write_count=0,
read_count=0)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=12)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=13)
asgn1_driver: ioctl ASGN1_GET_STATS executed (open=13, write=0, read=0, len=0,
last_write=17)
asgn1_driver: device closed
asgn1_driver: device opened (open_count=14)
asgn1_driver: ioctl invalid magic 0x5a (expected 0x41)
asgn1_driver: device closed
asgn1_driver: module unloaded cleanly
counter_driver: initializing miscellaneous device driver
counter_driver: loaded successfully on /dev/counter (minor=125)
counter_driver: read counter=0, auto-incremented
counter_driver: read counter=1, auto-incremented
counter_driver: read counter=2, auto-incremented
counter_driver: counter set to 100 via write
counter_driver: read counter=100, auto-incremented
counter_driver: read counter=101, auto-incremented
counter_driver: counter set to 0 via write
counter_driver: read counter=0, auto-incremented
counter_driver: negative value not allowed: -50
counter_driver: invalid non-numeric input: "abc_invalid"
counter_driver: module unloaded cleanly
```

[Optional Screenshot Space - IMG-02: IMG-02_task2_build_modinfo.png (Paste screenshot or image here if desired)]

Figure IMG-02: Compilation and modinfo verification output for counter_driver.ko on Ubuntu Host

4.5. Practical Troubleshooting and Debugging Analysis

During development and testing in the target environment, three critical technical challenges arose and were diagnosed and resolved:

1. Issue 1 — Kernel Stack Frame Size Overflow Warning (> 1024 bytes):
• Symptom & Warning: In the initial prototype of asgn1_write(), a local stack buffer char temp_buf[1024]; was declared to receive user data via copy_from_user(). Upon running make, the compiler issued: 'warning: the frame size of 1032 bytes is larger than 1024 bytes [-Wframe-larger-than=]'.
• Root Cause: Unlike user-space applications possessing megabytes of stack memory, each kernel execution thread on ARMv7 is allocated a fixed, highly constrained Kernel Stack (8KB). Consequently, Kbuild enforces a strict warning threshold restricting individual function stack frames to 1024 bytes to prevent catastrophic Kernel Stack Overflows (Kernel Panic).
• Resolution: Completely removed local buffer temp_buf[1024] from asgn1_write(). Instead, following mutex_lock(&asgn1_mutex), copy_from_user() copies user payloads directly into static buffer device_buffer residing in the module's .bss section. The compiler warning vanished completely.

2. Issue 2 — ARM Assembler Error with Memory Barrier Instruction 'isb' (Error: selected processor does not support `isb' in ARM mode):
• Symptom & Error: When compiling modules utilizing copy_to_user() / copy_from_user() with arm-linux-gnueabihf-gcc 15.2.0 on modern Ubuntu hosts, the Assembler threw errors: '/tmp/ccOaiuCn.s:613: Error: selected processor does not support `isb' in ARM mode'.
• Root Cause: Within Linux 5.15 source code (arch/arm/include/asm/uaccess.h), copy_from_user() inserts ISB (Instruction Synchronization Barrier) assembly instructions to flush instruction pipelines as a mitigation against speculative execution vulnerabilities (Spectre v1). ISB instructions require ARMv7-A architecture compliance. When compiling out-of-tree modules without explicit target flags, the cross-compiler defaulted to older ARM architectures when invoking the Assembler.
• Resolution: Explicitly injected 'ccflags-y := -march=armv7-a' into the Makefiles of both task1_ioctl and task2_misc modules, perfectly synchronizing instruction set targets with the ARM Cortex-A9 processor.

3. Issue 3 — Loss of Special Device Nodes (/dev/console, /dev/null) when Populating Rootfs without Privileges:
• Symptom: Utilizing standard cp -r commands to copy rootfs trees triggered: 'cp: cannot create special file .../dev/console: Operation not permitted'.
• Root Cause & Resolution: Nodes in /dev/ are Character Device Nodes containing Major/Minor inode metadata requiring CAP_MKNOD privileges. Resolved by utilizing 'sudo

cp -a' (archive mode) to preserve special file attributes, followed by chown to assign ownership back to the regular user account.

5. SECTION 5 (TASK 4): THEORY QUESTIONS AND ANSWERS (10 MARKS)

Question 1: Reference Counting (module_get / module_put) and rmmod Behavior when Devices are Open

• When does the Reference Count Increment and Decrement?

In the Linux kernel, every module is represented by a struct module containing an atomic reference counter (refcnt). By assigning '.owner = THIS_MODULE' within the driver's struct file_operations table, the Virtual File System (VFS) automatically manages module lifecycles:

- Increment (+1): Whenever a user-space process invokes open() on a device node (e.g., /dev/asgn1 or /dev/counter) or mounts driver resources, VFS invokes try_module_get(fops->owner) PRIOR to calling the driver's asgn1_open() routine.
- Decrement (-1): Whenever a process closes the final file descriptor via close() (or terminates, prompting the kernel to reclaim file descriptors), VFS executes asgn1_release() and subsequently calls module_put(fops->owner) to decrement refcnt by 1.

• What Occurs if rmmod is Attempted while a Process Keeps the Device Open?

When issuing rmmod, the kernel executes the delete_module() system call, inspecting the module's refcnt (observable in the 3rd column of lsmod). Because an open file descriptor exists, refcnt >= 1 (> 0). The kernel immediately rejects the removal request, returning -EBUSY while rmmod reports: 'rmmod: ERROR: Module asgn1_driver is in use'. This safeguard is critical: allowing module machine code to be evicted from RAM while a process maintains an active file handle would cause subsequent read/write/close calls to dereference dangling function pointers, instigating an immediate Kernel Oops or Panic.

• How to Handle or Force Unload the Module in Such Scenarios?

1. Standard Operational Practice: Identify holding processes via 'fuser -v /dev/asgn1' or 'lsof /dev/asgn1', then terminate those processes (kill <PID>) to force VFS file closure, dropping refcnt to 0 before executing rmmod.
2. Kernel-Level Force Unload: If compiled with CONFIG_MODULE_FORCE_UNLOAD=y, administrators may execute 'rmmod -f <module_name>'. However, forcing module removal with refcnt > 0 taints the kernel and risks system crashes if orphaned processes attempt future execution into unmapped driver addresses.

Question 2: Why copy_to_user() Must Be Used Instead of memcpy() in ASGN1_GET_STATS? MMU Level Mechanics

• Why memcpy() or Direct Pointer Dereference is Prohibited:

Within ASGN1_GET_STATS ioctl, the argument arg passed from user space is a User Virtual Address. Utilizing memcpy((void *)arg, &stats, sizeof(stats)) or direct pointer assignment

(*(struct asgn1_stats *)arg = stats) is strictly prohibited due to three fatal risks:

1. Invalid Pointers or Unmapped Memory (Demand Paging): User processes can pass NULL pointers (0x0), unmapped virtual addresses lacking physical RAM backing (due to Linux lazy allocation), or read-only memory regions.

2. Privilege Escalation Vulnerabilities: Malicious user applications could pass kernel-space virtual addresses as arg (e.g., >= 0xC0000000 on ARM 32-bit). Because kernel drivers execute in Supervisor Mode (SVC / PL1), memcpy() would blindly overwrite sensitive kernel data structures or page tables.

3. Hardware Access Protection (PAN / Domain Access Control): Modern ARM processors implementing PAN (Privileged Access Never) hardware bit restrictions forbid kernel-mode execution from accessing user-flagged memory pages unless temporarily unlocked via specialized routines inside copy_to_user().

• What Happens at the MMU Level if the Kernel Attempts Direct Access without copy_to_user()?

When CPU execution attempts memory stores, the Memory Management Unit (MMU) consults Page Tables (TTBR0/TTBR1 on ARMv7) to translate virtual addresses to physical addresses and validate Access Permission (AP) bits:

- If memcpy() encounters unmapped user pages or garbage addresses, the MMU fires a hardware Data Abort exception (Page Fault). The kernel's fault handler inspects the Program Counter (PC) and detects that the fault originated within kernel code (memcpy) absent from the Exception Table (__ex_table). The kernel classifies this as a severe programming bug, triggering a Kernel Oops or catastrophic Kernel Panic!

- Conversely, copy_to_user() initially executes access_ok(arg, size) to rigorously verify that the entire address range [arg, arg + size) resides strictly within User Space bounds (below TASK_SIZE). Second, internal assembly routines register their exception handling PCs in the '__ex_table' exception table. If an MMU Data Abort occurs due to swapped-out user pages or null pointers, the fault handler inspects __ex_table: if valid, it pages in memory and resumes; if illegal, it safely fixes up execution flow, returning -EFAULT to user space while the kernel remains entirely stable.

Question 3: Difference between MTD Subsystem and Block Device Subsystem in Embedded Systems

• Fundamental Difference between MTD (Memory Technology Device) and Block Device Subsystems:

The MTD subsystem is an abstraction layer within the Linux kernel engineered explicitly for raw flash memory chips, differing fundamentally from the Block Device subsystem designed for magnetic disks or storage media featuring integrated storage controllers:

1. Fundamental Physical Operations (3 Ops vs 2 Ops): Block Devices execute two core operations: Read sector and Overwrite sector. Conversely, raw flash memory managed by MTD cannot overwrite 0s to 1s directly, strictly enforcing the physical rule 'Erase-before-

Write' across three operations: Read, Write, and Erase.
2. Asymmetric Granularity: In Block Devices, read and write units are identical (Sectors of 512B or 4KB). In MTD (especially NAND Flash), read/write granularity is a Page (e.g., 2KB or 4KB), whereas the minimum erase granularity is a massive Eraseblock (e.g., 64KB, 128KB, or 256KB — as observed in vexpress-a9 dmesg: 'erase region 0: offset=0x0, size=0x40000 (256KB), blocks=256').
3. Bad Block Management, ECC, and Wear Leveling: Flash memory exhibits limited write/erase endurance (10,000 to 100,000 cycles for SLC) and develops Bad Blocks over time. Block Layer subsystems possess zero awareness of Bad Blocks or Error Correction Code (ECC); mounting traditional block filesystems (ext4, FAT) directly onto raw flash without wear leveling destroys storage blocks prematurely.

• When Must Embedded Storage Use MTD Instead of Block Layers? Concrete Examples:
Storage media must be managed via MTD when hardware consists of raw flash memory connected directly to processors via CFI parallel buses, SPI, or QSPI without an intervening Flash Translation Layer (FTL) controller. The Linux kernel couples MTD with specialized flash filesystems such as JFFS2, YAFFS2, or UBIFS (running atop UBI for wear leveling).
- Examples BINDING to MTD: Parallel NOR Flash chips (such as physmap-flash 64MB at 0x40000000 on vexpress-a9), SPI NOR Flash (e.g., Winbond W25Q128 16MB on embedded routers storing U-Boot/Kernel), and raw SLC/MLC NAND Flash (Micron MT29F).
- Conversely, storage media such as MicroSD cards, eMMC chips, NVMe SSDs, and USB Flash drives incorporate onboard microcontrollers running FTL firmware that abstracts bad blocks and emulates standard 512B/4KB sectors, thereby operating under the Block Device Subsystem.

Question 4: Handling of File Offset *ppos in read() and the Termination Loop of cat

• VFS read() Iteration Mechanism for 'cat /dev/counter' and Termination Conditions:
When executing 'cat /dev/counter' in user space, the cat utility has no prior knowledge of file or device size. Consequently, cat adheres to a standard polling loop:
1. Invokes fd = open("/dev/counter", O_RDONLY): The kernel allocates a struct file instance in the open file table, initializing file->f_pos = 0.
2. Read Loop: cat executes while ((n = read(fd, user_buf, BUF_SIZE)) > 0) { write(1, user_buf, n); }. Each read() system call passes file->f_pos into the 'loff_t *offset' (*ppos) pointer argument of counter_read().
3. Termination Condition (EOF - End Of File): Per POSIX standards, cat breaks its read loop and closes the descriptor exclusively when read() returns 0 (signaling EOF) or a negative error code.

• What Happens if counter_read() Fails to Validate and Update *ppos (*offset)?
In counter_driver.c, safety guard logic was implemented:
if (*offset > 0) return 0; /* Already read in previous invocation -> return EOF */

```
...
*offset += str_len;        /* Update file offset pointer following copy_to_user */
return str_len;
```

Execution mechanism: During the first read() call by cat, *offset equals 0, so the driver copies the string (e.g., "0\n", 2 bytes), advances *offset to 2, increments the counter, and returns 2. cat prints '0' and immediately re-enters its while loop, invoking read() a second time! In this second invocation, *offset equals 2 (> 0), triggering counter_read() to return 0 (EOF) instantly. Receiving 0, cat terminates cleanly after printing once.

Conversely, if the developer omits 'if (*offset > 0) return 0;' or fails to increment '*offset += str_len;':
During subsequent loop iterations, counter_read() restarts from the beginning, fetching new counter values, auto-incrementing, and continuously returning a byte count > 0 (never signaling EOF). The cat utility never receives an EOF notification, entering an INFINITE LOOP: the terminal scrolls tens of thousands of ascending numbers (0, 1, 2, 3, 4...) indefinitely, consuming 100% CPU until forcibly aborted via Ctrl+C (SIGINT)!